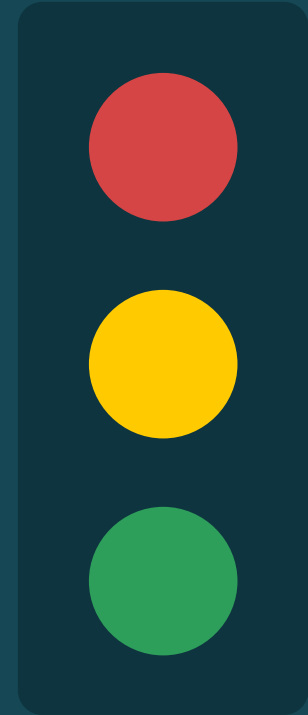


Feu piéton intelligent avec compte à rebours

Machine à états non bloquante sur Arduino Uno • Simulation Wokwi

GROUPE 1

Kassem Dehou Modeste • William Marrion Branham Ndala • Kra Junior
Mohamed Sow • Diouf François Pape • Saibou Kourouma



Le problème

Concilier la fluidité de la circulation et la sécurité du piéton

Sur un passage piéton, un feu à cycle fixe fait attendre les voitures même quand personne ne traverse. À l'inverse, un feu qui obéirait instantanément au bouton couperait la circulation sans prévenir et créerait un danger.

Le système doit donc être à la demande, mais rester maître du moment où il change les feux.

1

Fonctionnement à la demande

Les voitures passent par défaut ; le piéton signale son besoin par un bouton.

2

Séquence sécurisée

Orange, puis tout-rouge de sécurité avant ET après la traversée du piéton.

3

Robustesse garantie

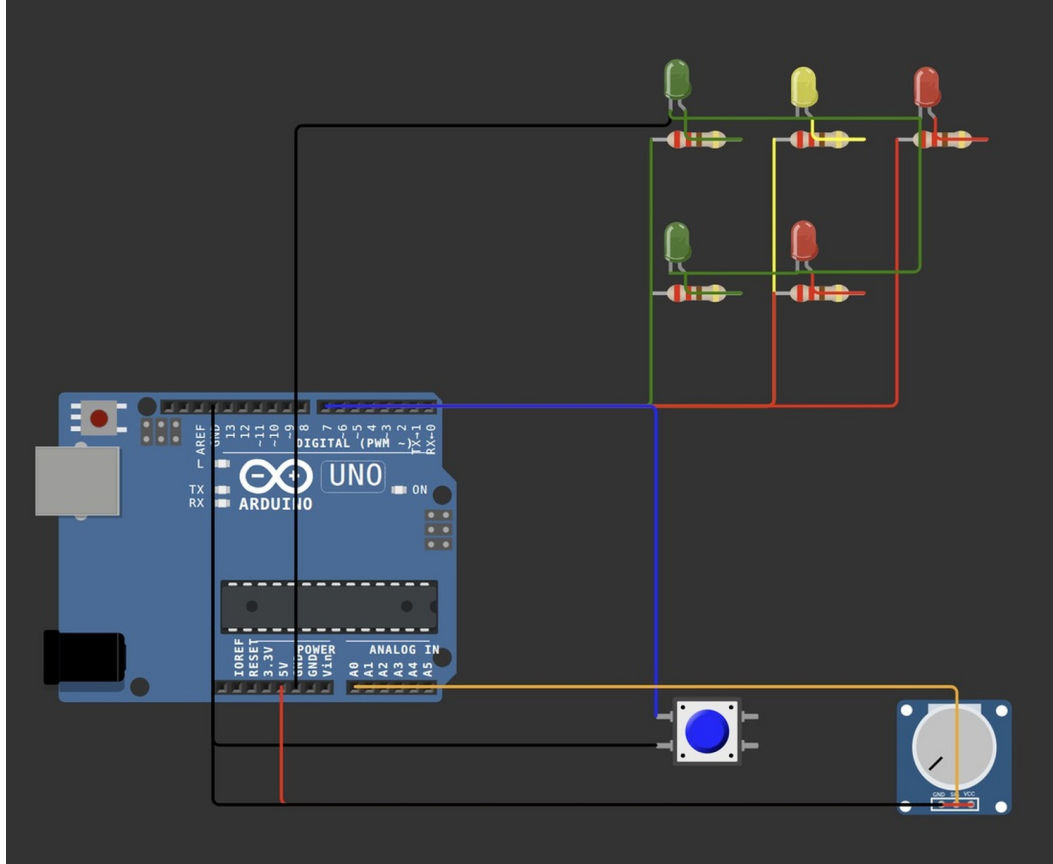
Un appui pendant la séquence ne doit ni la casser ni empiler une seconde demande.

Contraintes imposées

Entrées	2 minimum — bouton piéton + potentiomètre
Sorties	2 minimum — 5 LED pilotées séparément
Logique	Machine à états, 4 états minimum
Anti-rebond	Obligatoire sur le bouton piéton
Durées	Orange 2 s • Tout-rouge 1 s • Piéton 6 s
Compte à rebours	Affiché pendant la traversée, 6 → 0
Debug Serial	État + temps restant + événements
Câblage	Une LED = un signal, avec résistance

Schéma de câblage

Arduino Uno — 5 LED, 1 bouton, 1 potentiomètre



Règles respectées : une LED = un signal (aucune broche partagée) • chaque LED possède sa résistance 220 Ω en série • bouton lu de façon stable via INPUT_PULLUP + anti-rebond logiciel.

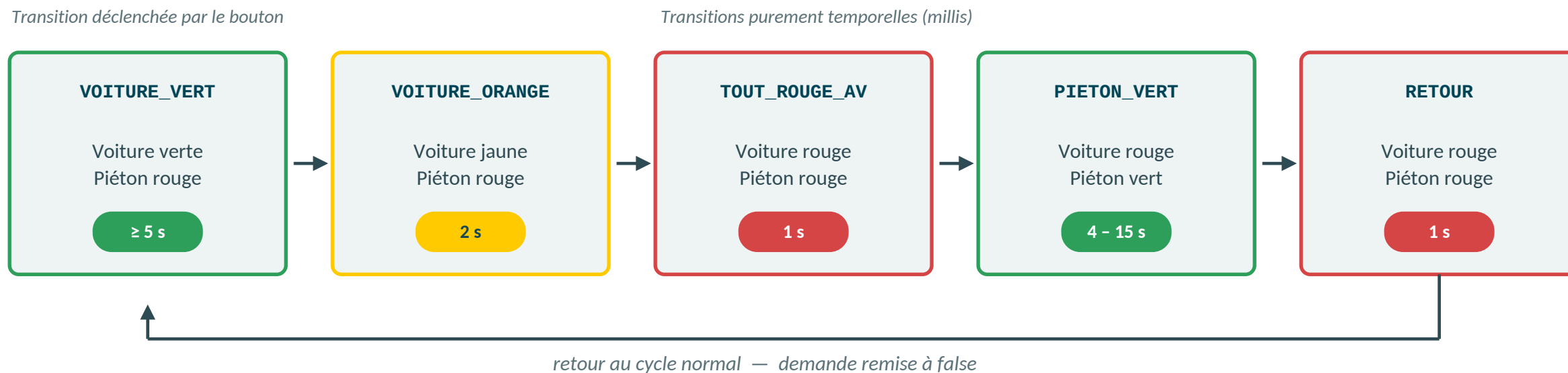
Brochage

- **D2** LED voiture VERTE
- **D3** LED voiture JAUNE
- **D4** LED voiture ROUGE
- **D5** LED piéton ROUGE
- **D6** LED piéton VERTE
- **D7** Bouton piéton → GND
- **A0** Potentiomètre (SIG)
- **GND** Cathodes + masse commune

Composants : Arduino Uno • 2 LED vertes • 1 LED jaune • 2 LED rouges • 5 résistances 220 Ω • 1 bouton poussoir • 1 potentiomètre

La logique : machine à états

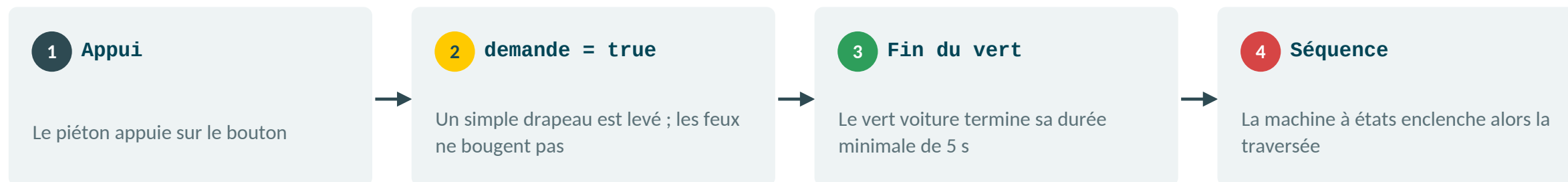
5 états — le sujet en exige 4, nous ajoutons le tout-rouge de sécurité avant la traversée



À tout instant le système se trouve dans exactement un état. Chaque état sait ce qu'il allume et à quelle condition il cède la place au suivant. Aucun `delay()` n'est utilisé : la boucle tourne en continu et reste donc capable de lire le bouton et de décompter en même temps.

Le bouton ne commande pas les feux

Il enregistre une intention que la machine à états consomme au bon moment



Pourquoi cette séparation ?

Si le bouton changeait l'état directement, la voiture passerait du vert au rouge sans orange ni tout-rouge. Le bouton propose, la machine décide : c'est ce qui rend le système impossible à casser.

Pas de double demande

Le drapeau n'est remis à false qu'à la toute fin du cycle, dans l'état RETOUR. Tant qu'on n'est pas revenu au vert, un nouvel appui n'ajoute donc rien et se trouve simplement ignoré.

Extrait — la ligne qui garantit la règle

```
case RETOUR:
    if (tempsDansEtat() >= T_TOUT_ROUGE) {
        demande = false;           // remise à zéro au bon endroit
        changerEtat(VOITURE_VERT);
    }
```

Robustesse : 4 mécanismes pour 2 exigés

Le sujet impose d'en traiter deux ; nous en avons implémenté quatre

1 Anti-rebond du bouton

Un bouton mécanique rebondit 5 à 20 fois par appui. Tout changement survenant moins de 50 ms après le précédent est ignoré.

`lireBouton()`

1 appui → 1 seul événement

2 Temporisation non bloquante

Aucun `delay()` dans la boucle. Toutes les durées sont mesurées avec `millis()`, ce qui permet de lire le bouton et de décompter simultanément.

`tempsDansEtat()`

Le compte à rebours l'exige

3 Filtrage de la mesure

Le potentiomètre est lu via une moyenne glissante sur les 5 dernières valeurs, ce qui élimine le bruit de conversion analogique.

`filtrerPot()`

Durée affichée stable

4 Verrouillage de la durée

La durée de traversée est figée à l'entrée de l'état piéton : on ne raccourcit jamais un temps de traversée déjà commencé.

`case TOUT_ROUGE_AV`

Argument de sécurité réel

Le cinquième mécanisme de la liste du sujet — l'hystérésis — ne s'applique pas ici : notre système ne compare aucune mesure continue à un seuil.

Comportement attendu, état par état

Une seule combinaison de feux est possible à chaque instant

ÉTAT	FEUX VOITURE	FEUX PIÉTON	DURÉE	SORTIE VERS
VOITURE_VERT	Vert allumé	Rouge allumé	≥ 5 s	Orange, si demande
VOITURE_ORANGE	Jaune allumé	Rouge allumé	2 s	Tout-rouge avant
TOUT_ROUGE_AV	Rouge allumé	Rouge allumé	1 s	Piéton vert
PIETON_VERT	Rouge allumé	Vert allumé	4 à 15 s	Retour
RETOUR	Rouge allumé	Rouge allumé	1 s	Voiture vert

Garde-fou de sécurité

La fonction qui pilote les LED éteint systématiquement les cinq sorties avant d'allumer celles de l'état courant. Il devient alors structurellement impossible d'avoir la voiture au vert et le piéton au vert en même temps, même en cas d'erreur ailleurs dans le programme.

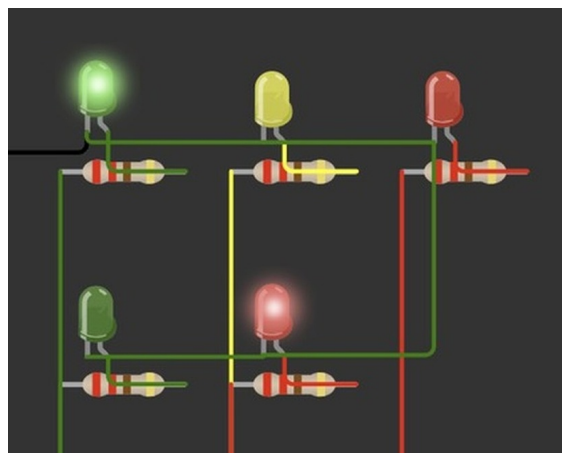
Sortie série attendue

```
[EVENEMENT] Demande pieton ENREGISTREE
[ETAT] --> VOITURE_ORANGE | demande=true
[EVENEMENT] Appui IGNORE (sequence en cours)
[ETAT] --> TOUT_ROUGE_AVANT
[ETAT] PIETON_VERT | Compte a rebours : 3 s
[ETAT] --> RETOUR | demande=false
```

Démonstration : les quatre états

Captures de la simulation Wokwi — une seule combinaison de feux à chaque instant

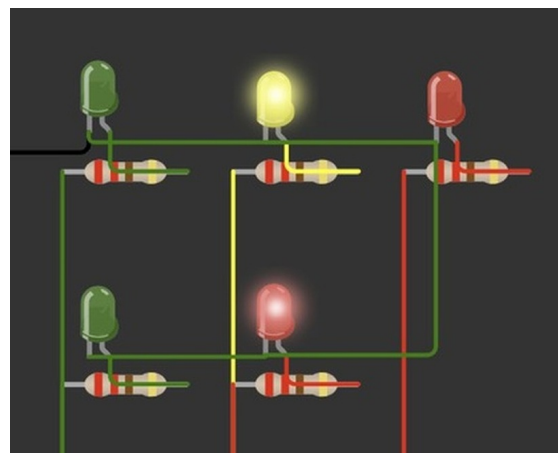
VOITURE_VERT



Voiture verte • Piéton rouge

Cycle normal : les voitures passent

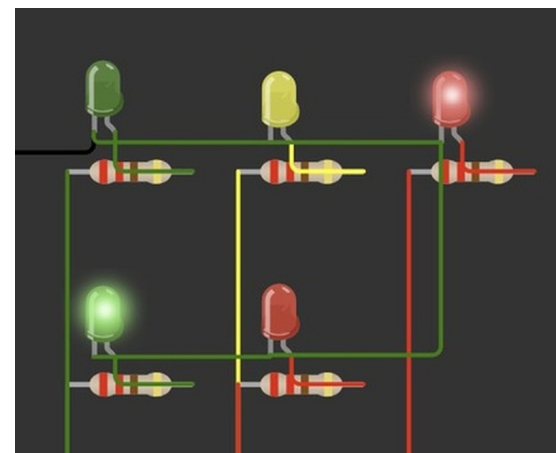
VOITURE_ORANGE



Voiture jaune • Piéton rouge

Transition après la demande

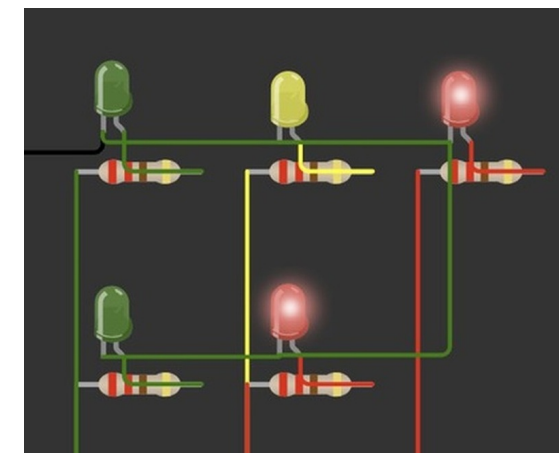
PIETON_VERT



Voiture rouge • Piéton vert

Traversée + compte à rebours

RETOUR



Voiture rouge • Piéton rouge

Tout-rouge de sécurité

À aucun moment la voiture et le piéton ne sont au vert en même temps. La fonction qui pilote les LED éteint les cinq sorties avant d'allumer celles de l'état courant, ce qui rend cette situation structurellement impossible — y compris en cas d'erreur ailleurs dans le programme.

Ce que nous avons testé

Validation du matériel d'abord, du logiciel ensuite

A

Câblage des 5 LED

Programme de test balayant les broches D2 à D6 une par une. Le nom affiché doit correspondre à la LED allumée.

B

Bouton et potentiomètre

Un appui produit une seule ligne : l'anti-rebond fonctionne. Le potentiomètre couvre bien la plage 4 – 15 s.

C

Séquence complète

Un appui déclenche orange, tout-rouge, piéton vert avec décompte jusqu'à 0, puis retour au vert.

D

Appui pendant la séquence

Le message Appui IGNORE apparaît et la séquence se poursuit sans double traversée.

Méthode : chaque bloc matériel a été validé par un programme de test dédié avant l'exécution du programme final, ce qui permet de distinguer immédiatement une erreur de câblage d'une erreur de logique.

```

=== FEU PIETON INTELLIGENT - DEMARRAGE ===
Etat initial : VOITURE_VERT
Appuyez sur le bouton bleu pour demander le passage.
[ETAT] VOITURE_VERT | Temps restant : 4 s | demande=false
[ETAT] VOITURE_VERT | Temps restant : 3 s | demande=false
[ETAT] VOITURE_VERT | Temps restant : 2 s | demande=false
[ETAT] VOITURE_VERT | Temps restant : 1 s | demande=false
[BTN] >> Demande piéton enregistrée (demande=true)
[ETAT] VOITURE_VERT | Temps restant : 0 s | demande=true

[ETAT] ==> VOITURE_ORANGE | demande=true
[BTN] >> Appui ignore (sequence en cours)
[ETAT] VOITURE_ORANGE | Temps restant : 1 s | demande=true
[ETAT] VOITURE_ORANGE | Temps restant : 0 s | demande=true

[ETAT] ==> TOUT_ROUGE_AVANT | demande=true
[ETAT] TOUT_ROUGE_AVANT | Temps restant : 0 s | demande=true
[INFO] Duree traversée fixée : 5 s (pot brut=125, filtre=125)

[ETAT] ==> PIETON_VERT | demande=true
[ETAT] PIETON_VERT | Compte à rebours : 5 s
[ETAT] PIETON_VERT | Compte à rebours : 4 s
[ETAT] PIETON_VERT | Compte à rebours : 3 s
[ETAT] PIETON_VERT | Compte à rebours : 2 s
[ETAT] PIETON_VERT | Compte à rebours : 1 s
[ETAT] PIETON_VERT | Compte à rebours : 0 s

[ETAT] ==> RETOUR (tout rouge) | demande=true
[ETAT] RETOUR (tout rouge) | Temps restant : 1 s | demande=true
[ETAT] RETOUR (tout rouge) | Temps restant : 0 s | demande=true

[ETAT] ==> VOITURE_VERT | demande=false
[ETAT] VOITURE_VERT | Temps restant : 4 s | demande=false
[ETAT] VOITURE_VERT | Temps restant : 3 s | demande=false
[ETAT] VOITURE_VERT | Temps restant : 2 s | demande=false

```

Moniteur série — cycle complet avec appui ignoré pendant la séquence

Bilan et démonstration

● 2 entrées minimum	Bouton piéton + potentiomètre	● 2 sorties minimum	5 LED pilotées séparément + moniteur série
● 3 modes minimum	5 états implémentés	● 2 robustesses	4 mécanismes implémentés
● Anti-rebond obligatoire	Filtre logiciel de 50 ms	● Compte à rebours	Décompte jusqu'à 0, une ligne par seconde
● Debug Serial	État, temps restant et événements	● Câblage conforme	Une LED = un signal, résistance en série

Simulation en ligne

<https://wokwi.com/projects/473342435632946177>